action object, very similar to how we used dispatch when calling a store in our coin example.

The only thing we are using the reducer for right now is to put the state and dispatch values into the context. This makes the reducer state and reducer dispatch function available to any component within the context.

Fetching Initial Data

Immediately after we call useReducer, we use the useEffect hook to fetch initial data from the server. I'd honestly rather have this fetch call be part of the reducer itself, but it's tricky to do asynchronous calls in a reducer; we'll see one solution to this problem in Chapter 12, *Using Redux to Manage State*. This code is similar to the useEffect call that we had in the Venue component and uses the same indirection to allow the asynchronous behavior. Instead of passing the resulting data to a useState setter, we're calling dispatch({ type: "setTickets", tickets: tickets }) to pass the data to the reducer—it's the same dispatch that was the return value from useReducer.

The server side changes slightly here to return the data as a list of ticket objects rather than a 2D array of them. We'll now handle conversion to the array client side. Here's the TicketController#index method that receives the API call:

chapter_11/01/app/controllers/tickets_controller.rb

```
def index
  @tickets = Ticket.data_for_concert(params[:concert_id])
  respond_to do |format|
    format.html
    format.json { render(json: @tickets) }
  end
end
```

And here's the method in the Ticket model that actually generates the data:

chapter_11/01/app/models/ticket.rb

```
def self.data_for_concert(concert_id)
  for_concert(concert_id).select(&:unavailable?).map(&:to_concert_h)
```